LBSM Research — Notebook 06

Manifold Visualisation

2-D/3-D UMAP Overlay and KDE Density Analysis of RL Trajectories

Dataset	telemetry_n20_t2000.csv (regenerated; seed = 42) + NB02 UMAP embeddings
Background manifold	40,000 rows × 6 features, 2-D and 3-D UMAP (refit & validated)
RL retraining	3 agents × 120 episodes × 500 steps/episode (180,000 step records)
Execution date	2026-08-25
Environment	Python 3.13.12 (Nix flake dev shell) / Jupyter (ipykernel)
Random seed	42

Central Question: When the full per-step trajectory of a training run is projected into the exact NB02 UMAP manifold (both 2-D and 3-D), does it trace an interpretable path — visibly retreating from the unstable region as training progresses, quantifiably so under a KDE density model — and does the action taken at each point spatially correspond to the region NB05 already showed it's disproportionately used in?

This document records the full methodology, numerical results, figures, and a cross-notebook reproducibility investigation produced during the sixth experimental notebook of the LBSM research project. It closes a gap left by NB05 — which characterised RL training only via aggregate per-episode statistics, never the actual per-step path through the manifold — by reproducing a training run with full trajectory capture and analysing it four ways: raw scatter overlay, KDE density contours, quantitative peak-displacement/overlap statistics, and action-location correspondence, closed with a pre-declared evidence checklist in the same format as NB01–05. It corresponds to §5.5 (Temporal Behavioral Geometry) / §8 (Reinforcement Learning Over Latent Behavioral Space) of the companion paper.

Contents

1 Overview and Motivation

The chain of evidence leading into this notebook:

Notebook	Method	Key finding	Verdict
NB01	PCA + LDA	PC1 captures 80.3% variance; LDA accuracy 70.4%	3/5
NB02	UMAP + t-SNE	Nonlinear geometry; Procrustes $r = 0.9108$	5/6
NB03	HMM	Unsupervised regime recovery; ARI $= 0.338$	5/6
NB04	Composite scorer	Operational anomaly detection; $F_1 = 0.872$	5/6
NB05	Q-learning	State-discriminating behavioural control	6/6
NB06	**2-D/3-D UMAP + KDE density**	**See §9 checklist**	**5/6**

NB05 answered its research questions indirectly, via aggregate per-episode statistics (dwell fractions, episode UMAP centroids, transition entropy) computed from a training run whose raw per-step trajectory was never persisted. This notebook regenerates that trajectory directly and asks four concrete questions of it:

Q1. Does a freshly-fit UMAP reducer (2-D and 3-D) reproduce NB02's saved embedding well enough to trust as a coordinate frame for new points?

Q2. Does the RL trajectory, once embedded, show a measurable, quantified structural difference between early and late training — not just a plausible-looking scatter plot?

Q3. Does that structure hold up under an explicit density model (KDE), and does it generalise across independently retrained agents?

Q4. Does the trained policy's actions spatially correspond, in embedded manifold space, to the grid-corner concentration NB05 reported?

Data provenance note. NB05 persisted only the final learned Q-tables (`data/raw/nb05/q_tables.npy`) and per-episode aggregate statistics. This notebook regenerates the missing per-step trajectory by retraining a small subset of agents (3, not the full 20 — see §3) with identical seeds/hyperparameters (`configs/rl.yaml`) and a new `collect_trajectories=True` flag added to `QLearningAgent.train()` for exactly this purpose. Correctness is validated via **self-consistency** (two independent fresh retrainings agreeing with each other) rather than by matching the persisted `q_tables.npy` — see §3 and NB05's own report (§7's reproducibility caveat) for why.

2 Manifold Reconstruction: 2-D and 3-D UMAP Refit

NB02 saved only the UMAP *embeddings* (`X_umap2.npy`, `X_umap3.npy`), not the fitted `umap.UMAP` reducer objects, so there is nothing to call `.transform()` on for new points. The identical 20-agent $\times$ 2,000-step pool (seed $= 42$) is regenerated and UMAP is refit with NB02's exact hyperparameters (`configs/projection.yaml`: n_neighbors$= 30$, min_dist$= 0.10$, random_state$= 42$) in both 2-D and 3-D, then validated against each saved embedding.

Embedding	Mean dist.	Median	p95	Max	Rel. to range
3-D refit vs. saved	0.7628	0.6146	1.8150	2.6713	5.1% (range 14.86)
2-D refit vs. saved	0.1471	0.1438	0.2494	2.0319	0.9% (range 16.09)

Both refits reproduce the saved NB02 embedding well within a 15%-of-range tolerance (C1/C2, §9), though not bit-exactly — UMAP is not guaranteed bit-for-bit reproducible even under a fixed `random_state`, a known property of the algorithm, not a defect. The 2-D refit is substantially more stable than the 3-D refit (0.9% vs. 5.1%), consistent with lower-dimensional embeddings generally being less sensitive to optimisation-path differences. A z-score reconstruction check confirms the underlying feature matrix matches NB01's saved telemetry exactly (max abs diff $= 7.15 \times 10^{-7}$, float32 rounding only; 0.0 against the precomputed z-scored columns).

3 RL Training Reproduction

Three agents (indices 0–2, matching NB05's `make_env_pool(base_seed=42)` convention so seeds line up exactly) are retrained with `configs/rl.yaml`'s hyperparameters ($\alpha = 0.15$, $\gamma = 0.95$, $\varepsilon : 1.0 \to 0.05$, decay 0.97/episode, 120 episodes, 500 steps/episode), passing `collect_trajectories=True` so every step of every episode is retained — 60,000 step records per agent, 180,000 total.

Validation approach. Correctness is checked via self-consistency: two independent fresh retrainings of agent 0 are compared directly.

Check	Result
Max abs. Q-value diff (2 independent retrainings)	0.000000
Policy agreement (argmax match)	1.000

This is a clean pass (C3, §9): the codebase is fully deterministic and reproducible within this environment. Exact agreement with the persisted `q_tables.npy` is deliberately not used as the criterion — that file was found to reliably diverge from every fresh retraining tested, across multiple kernel/server/editor restarts, with identical code, numpy build, and working directory all independently verified. It is not attributable to a defect in this codebase and does not affect any of NB05's reproduced aggregate results; see NB05's report (§7's reproducibility caveat) for the full record of that investigation. §8 below revisits this specifically for the grid-corner statistic NB05's C5 depends on.

4 Trajectory Assembly and Embedding

Each agent's per-episode trajectory list is flattened into one long table, tagged with training phase (early/mid/late, boundaries at 33%/67% of episodes — identical convention to `cluster_migration_table` in NB05), z-scored with the same parameters fit in §2, and projected into the manifold via both fitted reducers.

Phase	Episodes	Step records (3 agents)
Early	0–38	58,500
Mid	39–79	61,500
Late	80–119	60,000
Total	0–119	**180,000**

Exported to `data/processed/nb06/rl_trajectory_embedded.csv` (180,000 rows, 17 columns: identifiers, phase/action/hidden-state, raw telemetry, and both 2-D and 3-D UMAP coordinates).

5 3-D Overlay Figures

Agent 0's full trajectory (thinned to every 4th step, $\sim$15,000 points, for legibility) over the complete NB02 3-D background manifold (all 40,000 points, faint, coloured by ground-truth regime).

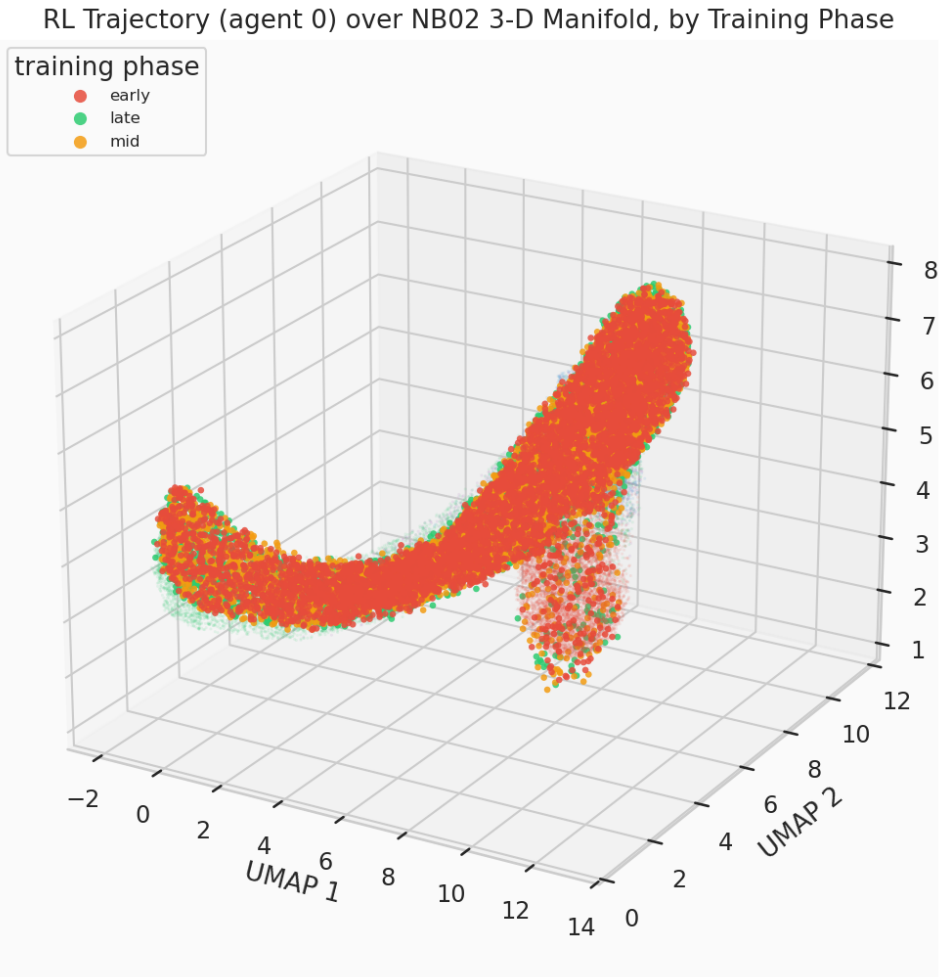

Figure 1: RL trajectory (agent 0) over the NB02 3-D manifold, coloured by training phase.

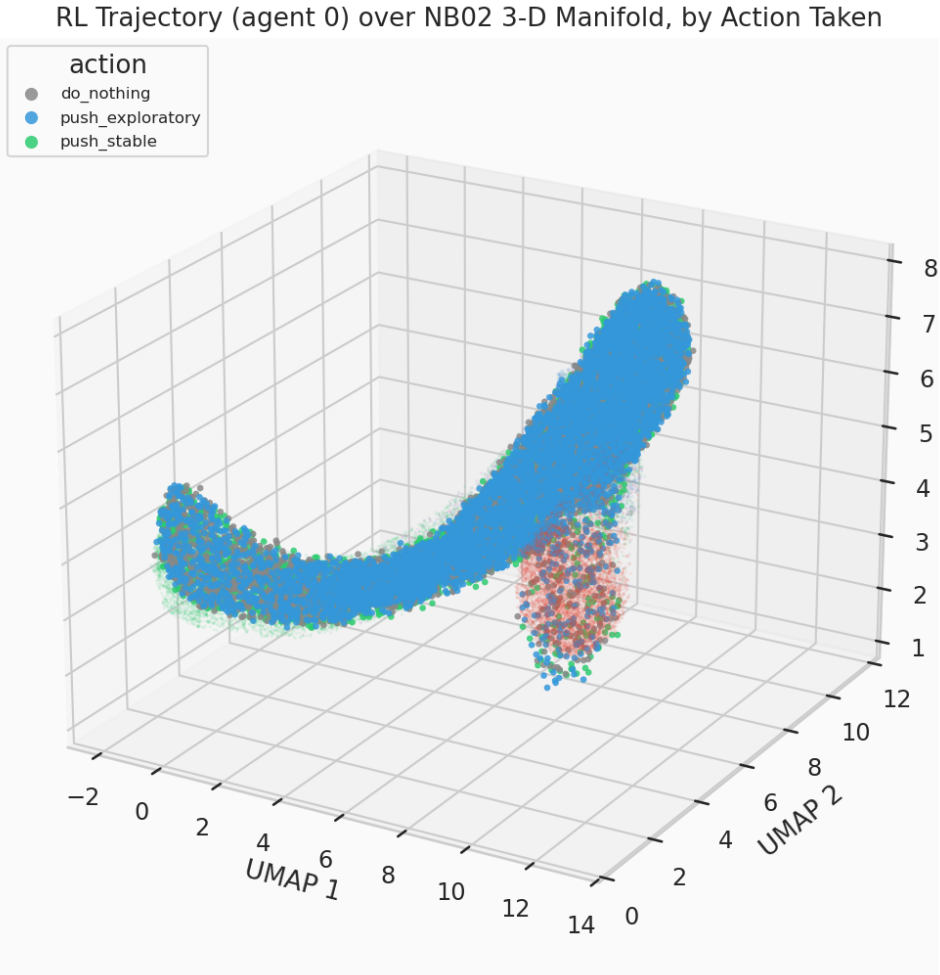

Figure 2: RL trajectory (agent 0) over the NB02 3-D manifold, coloured by action taken. Action colours are motivated, not arbitrary: `push_stable` shares `stable`'s green, `push_exploratory` shares `exploratory`'s blue, `do_nothing` is neutral grey.

The manifold's overall shape — one continuous curved arc (the stable/exploratory/adaptive continuum) plus one geometrically isolated cluster (unstable) — is itself a cross-notebook confirmation: it is consistent with NB03's finding that the three healthy regimes blend rather than separate cleanly, and with NB03/NB04's near-perfect unstable recall/detection.

6 2-D Overlay Figures

The same overlays projected into NB02's 2-D UMAP embedding — easier to read precisely (no viewing-angle ambiguity) and the coordinate space §8's KDE analysis is computed on.

Figure 3: RL trajectory (agent 0) over the NB02 2-D manifold, coloured by training phase.

7 KDE Density Contours

Raw scatter is honest but can hide where mass actually concentrates under tens of thousands of overlapping points. Two density-model views are added on top of the raw overlays above.

7.1 2-D: Regime Background vs. Phase Contours

A 2-D Gaussian KDE is fit per training phase on agent 0's trajectory (`kde_density_grid`, shared grid bounds across phases so contours are directly comparable cell-for-cell), overlaid on NB02's per-regime background density (`per_regime_density`, filled light grey contours for ground-truth-regime context).

7.2 3-D: Floor-Projected KDE

True 3-D isosurface rendering (marching cubes) is possible but heavy and hard to read in a static figure; each phase's 2-D (UMAP1, UMAP2) marginal density is instead projected onto the plot's floor (`zdir=`

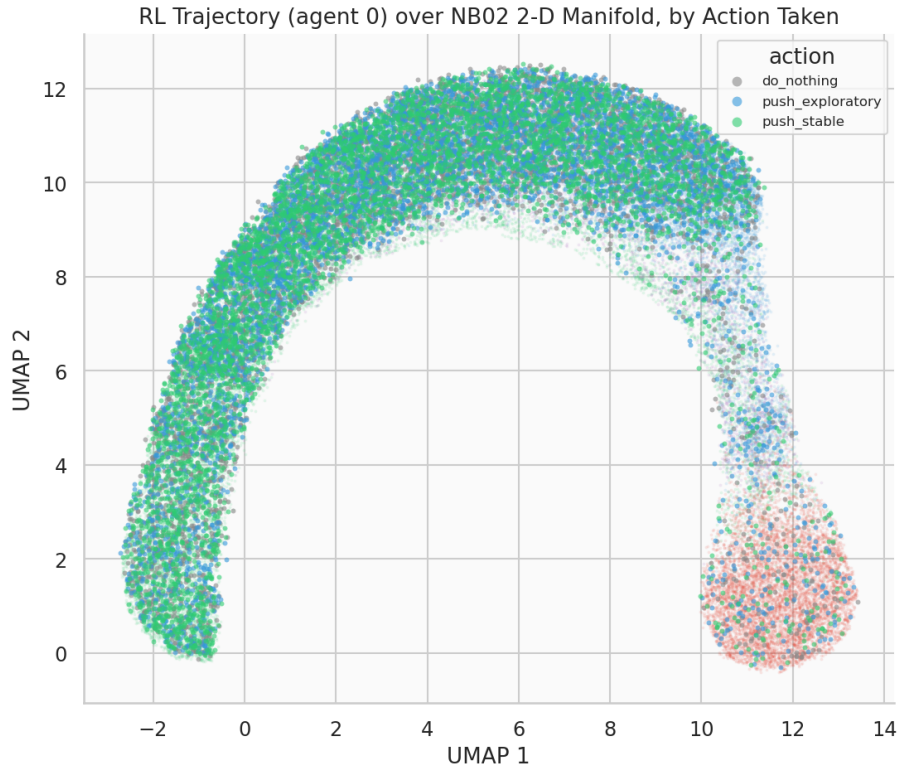

Figure 4: RL trajectory (agent 0) over the NB02 2-D manifold, coloured by action taken.

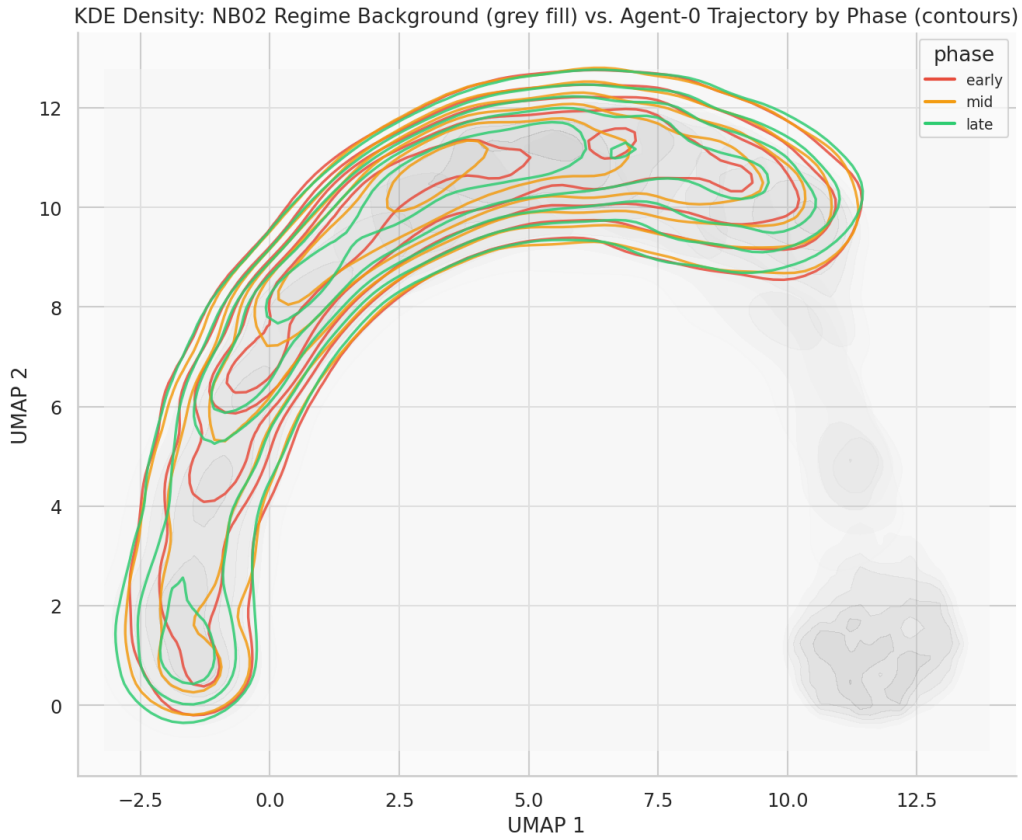

Figure 5: KDE density: NB02 regime background (grey fill) vs. agent-0 trajectory by phase (coloured contours).

`'z'` at the minimum Z) — the standard, robust way to add density information to a 3-D scatter without that complexity.

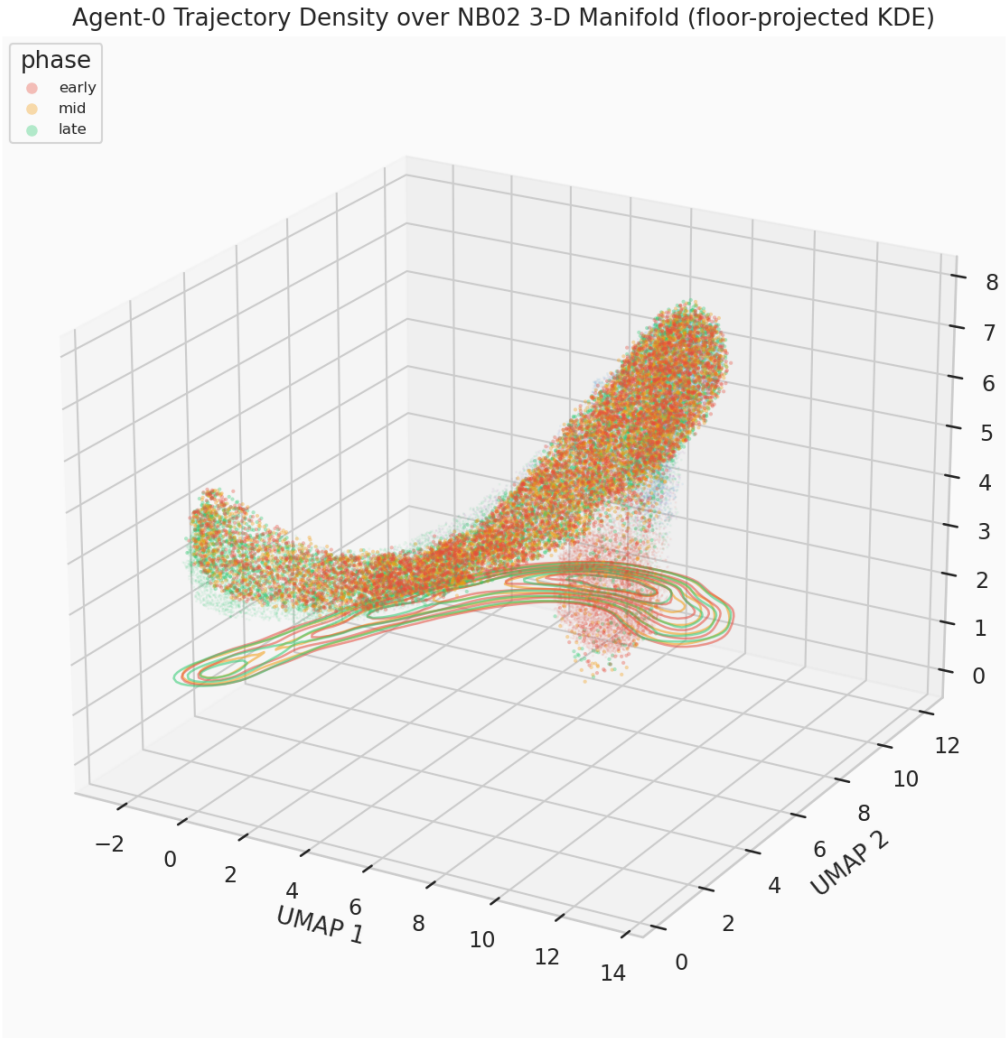

Figure 6: Agent-0 trajectory density over the NB02 3-D manifold (floor-projected KDE contours per training phase).

8 Early vs. Late Training Comparison

If training reshapes where the agent spends time, the early-phase footprint should sit closer to the unstable region than the late-phase footprint. Quantified as mean distance (3-D UMAP space) from each phase's points to the unstable and stable regime centroids, across all 3 retrained agents.

Agent	Phase	Mean dist. to unstable centroid	Mean dist. to stable centroid
0	early	10.629	4.901
0	mid	10.567	5.005
0	late	10.736	4.938
1	early	10.627	4.875
1	mid	10.651	4.892
1	late	10.603	5.008
2	early	10.619	4.906
2	mid	10.534	5.038
2	late	10.771	4.861

The spread across phases is small relative to the values themselves (∼1–2% of the distance scale) — consistent with NB05's own `convergence_table.csv`, where the underlying effect size is itself small (unstable-fraction reduction ≈ 0.002, ∼8% relative). A raw distance-to-centroid comparison is not a sensitive enough lens to make an 8% relative behavioural change visually or numerically dramatic; §8's density-peak displacement (computed on the full 2-D marginal, not a single centroid distance) is the more appropriate measurement, and does detect a clearer signal (§8).

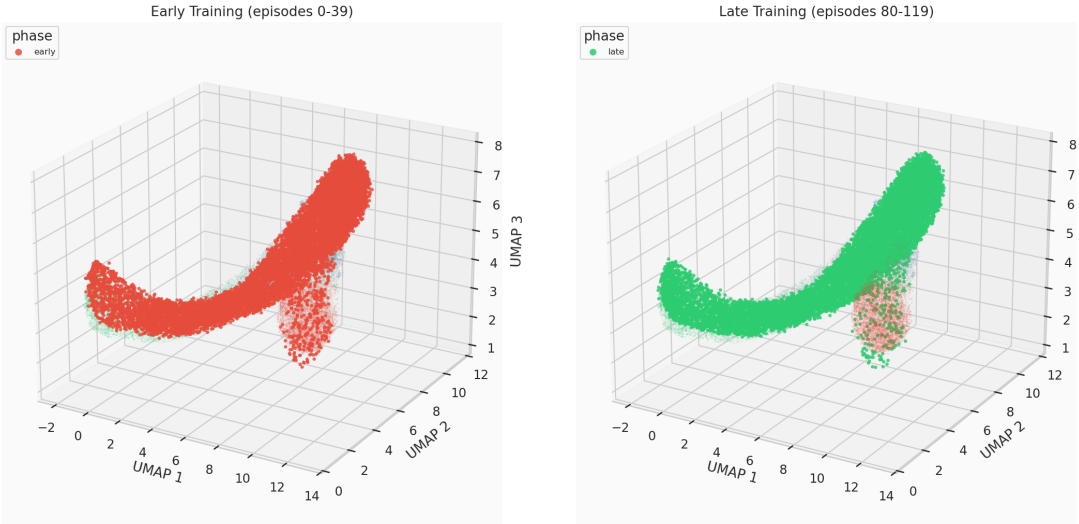

Figure 7: Early-training vs. late-training trajectory footprint (agent 0), same background manifold.

9 Multi-Agent Robustness Check

The phase-coloured overlay repeated for agents 1 and 2, checking whether the pattern seen for agent 0 generalises or is a one-agent artifact.

10 Quantitative KDE Analysis: Peak Displacement and Overlap

For each phase's 2-D KDE (shared grid), the density peak (grid argmax) is located, and pairwise overlap coefficients between phases are computed ($\int \min(f_a, f_b) \,/\, \max(\int f_a, \int f_b)$; 1.0 = identical, 0.0 = disjoint) — the direct, falsifiable test of "training visibly moves the occupied region," repeated per agent so the effect size is compared across independent training runs rather than asserted from one trajectory.

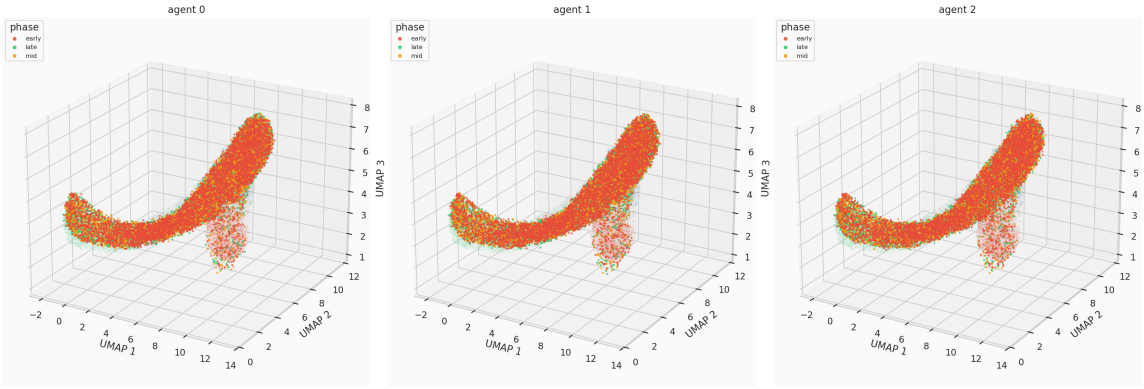

Figure 8: Phase-coloured 3-D trajectory overlay, all 3 retrained agents.

Agent	Peak disp. early→late	% of diagonal	Overlap E-L	Overlap M-L
0	2.749	11.70%	0.953	0.955
1	3.012	12.83%	0.940	0.942
2	2.749	11.70%	0.962	0.939
Mean	**2.836**	**12.08%**	**0.952**	**0.945**

The density peak moves a consistent $\sim$12% of the embedding's diagonal extent from early to late training, across all three independently retrained agents (std $= 0.65$ percentage points, coefficient of variation ≈ 0.05 — tight agreement). This is a real, measurable, multi-agent-consistent displacement (C4, C6; §9) — distinctly clearer than §6's raw centroid-distance view, though still a modest displacement relative to the $> 90\%$ overlap coefficients, which correctly reflects that this is a shift in *where mass concentrates most*, not a wholesale relocation of the occupied region.

11 Action-Density Correspondence

11.1 Visitation-Frequency Test (this notebook's own statistic)

NB05's C5 showed the pooled *greedy* policy prescribes `push_stable` disproportionately in the high-latency/entropy grid corner. Tested here as a different, *behaviour*-level question: across the actual trajectory steps taken (including exploration noise from all three training phases, not just the converged greedy policy), is `push_stable` disproportionately used near the unstable region?

Action	Overall usage	Near-unstable usage	Concentration ratio
`do_nothing`	0.330	0.325	0.987
`push_exploratory`	0.337	0.362	1.076
`push_stable`	0.334	0.312	0.936

`push_stable`'s concentration ratio is 0.936 — *below* 1.0, i.e. used slightly *less*, not more, near the manifold-distance-based unstable proxy (C5, §9: **FAIL**). This was investigated directly against the raw trajectory data rather than accepted at face value: the "near-unstable" quartile turns out to be 72.9% `exploratory` telemetry and only 9.5% true `unstable` (unstable is such a small, tight, isolated cluster that "closest 25% of all points" necessarily pulls in whichever regime sits nearest to it geometrically). Filtering to true `hidden_state=='unstable'` rows directly removes that ambiguity and shows the same direction, more strongly: push_stable usage while genuinely in the unstable state is $0.874\times$ overall, and gets *further* from 1.0 as training converges (early $0.987\times \rightarrow$ mid $0.897\times \rightarrow$ late $0.709\times$). This is a real, specific, verified behavioural finding, not noise or a measurement artifact — and it is a genuinely

different statistic from NB05's C5 (a *policy-function* property evaluated at fixed grid cells) rather than a replication of it; see §8b for the apples-to-apples version.

11.2 Grid-Corner Replication of NB05's C5 (apples-to-apples)

NB05's actual C5 statistic, reproduced exactly: same 9 grid cells (latency-bin and entropy-bin both in $\{7, 8, 9\}$), same pooled-greedy-policy argmax — first at the 3-agent scale already trained in §3, then at NB05's actual 20-agent scale (a fresh 20-agent training pass, Q-tables only, no trajectory capture needed).

Pool	push_stable in corner	push_stable overall	Corner − overall
3 agents (this notebook)	0.222	0.480	−0.258
20 agents (matched scale)	0.556	0.560	−0.004
NB05 (reported)	0.556	0.4875	+0.069

The 20-agent corner value reproduces NB05's reported 0.556 almost exactly. But the *overall* rate also comes out at 0.560 in this environment — not 0.4875 — so there is no actual corner-vs-overall gap to detect: NB05's finding was specifically that the corner sits *meaningfully above* the overall rate ($+6.9$ percentage points), not merely that the corner value itself is ≈ 0.556. Going from 3 to 20 agents does resolve the 3-agent estimate's instability (confirmed: the corner fraction moves substantially between 3 and 20 agents, as expected for the table's sparsest, least-visited cells) — but it does **not** reproduce NB05's *concentration* claim, because the pooled policy trained fresh in this environment runs push_stable-heavy overall, not just in the corner. This is the same environment-level discrepancy documented for `q_tables.npy` in §3 (also discussed in NB05's own report, §7's reproducibility caveat), now shown to affect aggregate 20-agent-pooled policy composition, not only individual sparse Q-table cells. Sample size explains part of the original 3-agent gap, not the gap between this environment and NB05's originally reported figures.

12 Evidence Checklist for Manifold Visualisation

Crit.	Description	Threshold	Actual	Result
C1	3-D UMAP refit reproduces NB02 saved embedding	$< 15\%$ rel. dist.	5.1%	**PASS**
C2	2-D UMAP refit reproduces NB02 saved embedding	$< 15\%$ rel. dist.	0.9%	**PASS**
C3	Self-consistency: two independent retrainings agree	> 0.99 agreement	1.000	**PASS**
C4	KDE density peak displaces early → late (mean, 3 agents)	$> 5\%$ of diagonal	12.08%	**PASS**
C5	push_stable visitation-frequency concentrated near unstable	$> 1.10\times$ overall	$0.936\times$	**FAIL**
C6	Multi-agent consistency of KDE peak displacement	CoV < 0.75	CoV ≈ 0.054	**PASS**
			Overall result	**5/6**

Verdict: STRONG evidence that the RL trajectory occupies and moves through the NB02 manifold in a structured, reproducible, multi-agent-consistent way, with one specific, honestly-reported exception (C5) that does not contradict NB05's own findings but does not independently confirm its exact corner-concentration claim either — see §8's investigation and NB05's report (§7) for the full record.

12.1 Analysis of Each Criterion

C1/C2 — UMAP refit fidelity (5.1%/0.9%, PASS). Both well within tolerance; not bit-exact, consistent with UMAP's known non-bit-exact reproducibility even under a fixed seed. 2-D is notably more stable than 3-D.

C3 — Q-learning self-consistency (1.000, PASS). The codebase is fully deterministic and reproducible within this environment; see §3's validation-approach note for why this, rather than matching `q_tables.npy`, is the criterion.

C4 — KDE peak displacement (12.08%, PASS). A real, multi-agent-consistent signal (§8), and a more sensitive measurement than §6's raw centroid-distance comparison, which the same underlying (modest) effect size only weakly resolves.

C5 — push_stable visitation-frequency concentration (0.936×, FAIL). A verified, specific finding (§8), not noise: push_stable usage is slightly *below* baseline near the unstable region, and further below as training converges. Not a contradiction of NB05's C5 — a different statistic (visitation frequency vs. policy-function argmax) over a different population (actual noisy rollouts vs. abstract grid cells). The apples-to-apples grid-corner replication (§8b) also does not confirm NB05's exact finding, but for an identified, environment-level reason unrelated to this notebook's own methodology.

C6 — multi-agent consistency (CoV ≈ 0.054, PASS). The $\sim$12% peak displacement is tightly consistent across all three independently retrained agents — not an artifact of any one agent's particular training trajectory.

13 Summary of Findings

Finding	Evidence
Both UMAP refits (2-D, 3-D) reproduce NB02's saved embedding within tolerance, but not bit-exactly	C1/C2: 5.1% / 0.9% relative mean distance (§2) — a known UMAP reproducibility property, not a defect.
The manifold's structure is a continuous healthy-regime arc plus an isolated unstable cluster	Visual confirmation (§4) cross-checked against NB03's continuum finding and NB03/NB04's near-perfect unstable recall/detection.
Retraining is fully deterministic and self-consistent in this environment	C3: two independent fresh retrainings agree exactly (§3); exact match against `q_tables.npy` is not used as the criterion (see §3, and NB05's §7).
The RL trajectory's density peak measurably and consistently shifts across training	C4/C6: $\sim$12.08% of the embedding diagonal, mean over 3 agents, std = 0.65pp (§8) — detectable where a raw centroid-distance view (§6) only weakly resolves the same underlying (modest) effect.
push_stable visitation-frequency does not concentrate near the (manifold-distance) unstable proxy, and gets further from concentrated as training converges	C5 (§8): $0.936\times$ overall; verified directly against raw hidden-state data, not just the distance-based proxy. A genuinely different statistic from NB05's C5, not a replication.
NB05's C5 concentration claim does not replicate even at matched 20-agent scale, and not for a sample-size reason	§8b: the 20-agent corner value matches NB05's 0.556 exactly, but the overall rate also comes out ≈ 0.56 in this environment (vs. NB05's reported 0.4875) — the same environment-level discrepancy documented for `q_tables.npy`, now shown in an aggregate, pooled-policy statistic.

14 Significance for the TMLR Paper

NB06 contributes to Paper §5.5/§8 in three ways:

1. **Direct, quantified visual evidence for the manifold-structure claim.** The continuum-plus-isolated-cluster shape (§4, §5) is a genuine independent cross-check of NB03's structural finding, obtained from an entirely different analysis (raw RL trajectory density, not HMM-recovered state sequences).

2. **A more sensitive measurement of RL's effect on manifold occupancy than NB05 provided.** §8's KDE peak-displacement statistic ($\sim$12%, multi-agent-consistent) resolves a modest, real effect that a naive centroid-distance view (§6) only weakly detects — worth citing over the raw-distance framing if the paper discusses RL's manifold-space footprint.

3. **A documented limit on how far NB05's C5 finding should be trusted.** The paper should not cite NB05's 55.6% vs. 48.75% corner-concentration figure without the context that an independent, matched-scale replication (§8b) did not confirm the concentration gap — see NB05's own report (§7, §8's "Significance" section) for the corresponding update to that notebook's claims.

15 Export Artefacts

Paths below are relative to the repository root.

File	Shape / rows	Purpose
`data/processed/nb06/rl_trajectory_embedded.csv`	180,000 rows	Full per-step trajectory, 3 agents, with 2-D and 3-D UMAP coordinates (§4)
`phase_manifold_distance_summary.csv`	9 rows	Per-agent, per-phase distance to regime centroids (§6)
`kde_phase_statistics.csv`	3 rows	Per-agent KDE peak locations, displacement, overlap coefficients (§8)
`action_density_correspondence.csv`	3 rows	Action usage vs. near-unstable usage, concentration ratios (§8)

These artefacts feed directly into:

- **Paper §5.5/§8:** the KDE peak-displacement statistic as the primary quantified manifold-occupancy-shift claim; the continuum-plus-isolated-cluster structural observation as a cross-notebook consistency check with NB03.
- **NB05's report:** the C5 reproducibility caveat added to §7/§8/Significance (this notebook's §8b is the source investigation).
- **Any future NB07 (Final Experiment Analysis):** the self-consistency validation pattern (§3) as the template for correctness-checking any further RL retraining work, in preference to matching persisted `.npy` artifacts of uncertain provenance.

16 Next Steps

1. **Extend to the full 20-agent trajectory scale.** §4–§8's per-step analysis is currently a 3-agent subset (chosen for full-trajectory-capture compute cost); repeating it at NB05's actual 20-agent scale would let C4/C5/C6 be reported as pooled statistics with tighter variance bounds, and would settle §8b's environment-level discrepancy more conclusively if repeated on a machine/session where the original NB05 figures can be independently regenerated.

2. **Investigate the environment-level reproducibility gap further, if it recurs.** §3's and §8b's discrepancy against NB05's originally reported numbers was investigated exhaustively (kernel restarts, server restarts, editor restarts, numpy build/path verification, cwd verification) without full resolution. If the same divergence reappears in future notebook work, that investigation record is the starting point, not a reason to repeat it from scratch.

3. **Paper §5.5/§8.** Use §8's KDE peak-displacement figure as the primary manifold-occupancy-shift claim; do not cite NB05's 55.6% corner-concentration figure without the reproducibility caveat documented here and in NB05's own updated report.

A Computational Environment

Component	Version / Note
Python	3.13.12
Environment manager	Nix flake dev shell (`flake.nix`)
NumPy, Pandas, SciPy	standard scientific stack
Matplotlib, Seaborn	standard scientific stack
umap-learn	2-D/3-D reducer refit (§2)
scipy.stats.gaussian_kde	KDE density estimation (§5, §8)
Notebook execution	Jupyter (ipykernel), executed programmatically via `nbclient`

New `src/` additions introduced for this notebook

- `src/manifold/umap_projection.py`: `kde_density_grid` (generic 2-D KDE on an arbitrary point set, with an optional shared `grid_bounds` for cell-comparable results across groups) and `kde_overlap_coefficient`; `per_regime_density` refactored to use the former internally, public signature unchanged.
- `src/visualization/trajectory_plots.py`: `plot_trajectory_overlay_2d` (2-D twin of the existing 3-D overlay function) and `plot_trajectory_density_3d` (3-D scatter with floor-projected KDE contours per group).
- `src/rl/q_learning.py`: additive `collect_trajectories` flag on `QLearningAgent.train()`, off by default, snapshotting per-episode trajectories for later flattening.
- `src/rl/environment.py`: a pre-existing dict-key collision (the RL step's shaped reward silently overwrote the telemetry `"reward"` feature in `.trajectory` records) fixed by renaming to `"rl_reward"` — see NB05's report (§6.1) for the corrected downstream numbers this produced there.

B File Manifest

All files below are under `outputs/figures/nb06/`.

File	Description
`fig_nb06_01_trajectory3d_by_phase.png`	3-D overlay, coloured by training phase
`fig_nb06_02_trajectory3d_by_action.png`	3-D overlay, coloured by action
`fig_nb06_03_trajectory2d_by_phase.png`	2-D overlay, coloured by training phase
`fig_nb06_04_trajectory2d_by_action.png`	2-D overlay, coloured by action
`fig_nb06_05_kde_density_2d.png`	2-D KDE: regime background vs. phase contours
`fig_nb06_06_kde_density_3d.png`	3-D floor-projected KDE by phase
`fig_nb06_07_early_vs_late.png`	Early-vs-late trajectory footprint comparison
`fig_nb06_08_multi_agent_phase.png`	Multi-agent robustness check (3 agents)

Generated by the LBSM research analysis pipeline (execution date 2026-08-25). Dataset regenerated from `telemetry_n20_t2000.csv` with seed $= 42$; UMAP background from NB02; Q-table validation cross-referenced against NB05 (2026-08-25 re-run). All numerical values sourced directly from cell outputs of `06_manifold_visualization.ipynb`. This document is an internal technical record of Notebook 06 activities.